

CS 12 Sample 1

Department of Computer Science
College of Engineering
University of the Philippines Diliman

1 Instructions

1. READ **ALL** OF THESE INSTRUCTIONS **VERY VERY VERY CAREFULLY!!!** 😊

- You may lose points for not following any of the instructions.
2. Answer problems in their respective answer boxes.
 - If the answer box doesn't fit your answer, you may *continue* at the back of the **corresponding page** (**not** on a different page), but say so in the corresponding answer box.
 - Do not remove/replace the label identifying the question an answer box corresponds to.
 - Answers not following the above rules **will not be marked**.
 3. Place your bags in the front portion of the exam room, place all electronic communication devices in your bags, and make sure to set them to **silent mode**.
 4. Submit *all* answer sheets (even blank ones), your questionnaire, and all your scratch papers.
 - Make sure your name *and* student number are at the top of *each* answer sheet (even blank ones), front side.
 - Write your name, student number and section (or discussion class date/time) on the *upper-right-hand corner of the first page* of the questionnaire.
 - Write your name on every scratch paper you used.
 - You may write on the questionnaire. (They will not be considered part of your answers.)
 - Do **not** remove staple wires. (You may remove the removable fasteners.)
 5. Use a **black or blue pen** for all solutions, answers, and other written requirements. Pencils, any form of erasable ink, and red ink are **not** allowed. Answers not following this rule **will not be marked**.
 6. Write clearly and legibly. This includes writing in *complete sentences*, especially for proving and essay questions. If the answers are unclear, they may be *ignored*.
 7. For items with computations, show *complete* solutions, unless it is stated that solutions are not required. Answers with incomplete/lacking solutions will likely be given few to no points.
 8. Make sure that there is no missing page in this questionnaire.

Above all else, your goal is to *communicate*. After all, if a grader cannot understand your solution, they cannot give you any credit for it.

The exam will be scored over **1000 points**.

Exam Problems

[120 points] 1. The following are different declarations of a function named `consume`.

Declaration A:

```
def consume(x: list[int]):
    ...
```

Declaration B:

```
def consume(x: tuple[int, int]):
    ...
```

Declaration C:

```
def consume(x: Iterable[int]):
    ...
```

Declaration D:

```
def consume(x: Sequence[int]):
    ...
```

Declaration E:

```
def consume(x: MutableSequence[int]):
    ...
```

Declaration F:

```
def consume(x: object):
    ...
```

The following are different usages of `consume`: (See [Appendix C](#) in case you are not familiar with `tuple[T, ...]`.)

Usage 1:

```
def produce() -> tuple[int, ...]:
    return (1, 2)
```

Usage 2:

```
def produce() -> tuple[int, int]:
    return (1, 2)
```

Usage 3:

```
def produce() -> tuple[int, ...]:
    return (1, 2, 3)
```

Usage 4:

```
def produce() -> tuple[int, int]:
    return (1, 2, 3)
```

Usage 5:

```
def produce() -> list[int]:
    return [1, 2]
```

Usage 6:

```
def produce() -> object:
    return (1, 2)
```

Usage 7:

```
def produce() -> Sequence[int]:
    return (1, 2)
```

Usage 8:

```
def produce() -> MutableSequence[int]:
    return (1, 2)
```

Usage 9:

```
def produce() -> Sequence[int]:
    return [1, 2]
```

Usage 10:

```
def produce() -> MutableSequence[int]:
    return [1, 2]
```

For which pairings of function declaration and usage will the call `consume(produce())` type-check correctly? Format your answer as a table like so (where each “?” is either “yes” (if it will type-check) or “no” (if it will not type-check)):

	1	2	3	4	5	6	7	8	9	10
A	?	?	?	?	?	?	?	?	?	?
B	?	?	?	?	?	?	?	?	?	?
C	?	?	?	?	?	?	?	?	?	?
D	?	?	?	?	?	?	?	?	?	?
E	?	?	?	?	?	?	?	?	?	?
F	?	?	?	?	?	?	?	?	?	?

Note that for this item, incorrect answers will give point deductions! (A missing answer is not considered incorrect.) More specifically:

- leaving a cell blank gives you 0 points;
- putting in the correct answer (yes or no) gives you 2 points;
- putting in an incorrect answer gives you –3 points.

2. Shiritori is a game played by two or more players. In this game, an initial word is given, and the players take turns giving words. The following rules must be followed throughout the game.

- The word given by a player must start with the last letter of the previous word given.
- The word must belong to a list of valid words, fixed at the beginning of the game.

A naive implementation of Shiritori is given in Code Block A. (See [Appendix B](#) in case you are not familiar with the “:=” syntax.)

Code Block A Code for item 2

```
class Shiritori:
    def __init__(self, valid_words: Iterable[str]):
        self.valid_words: list[str] = list(valid_words)
        self.previous_word: str | None = None
        self.word_count: int = 0
        super().__init__()

    def prompt_word(self) -> str:
        word = input('Please enter a word: ').rstrip().lower()
        if not word:
            self.print_feedback('Enter a word with at least one letter')
            return None
        elif word not in self.valid_words:
            self.print_feedback(f'The word "{word}" is not in the list of valid words.')
            return None
        return word

    def print_feedback(self, feedback: str):
        print(feedback)

    def play_word(self, word: str) -> bool:
        if self.previous_word is None or self.previous_word[-1] == word[0]:
            self.previous_word = word
            self.word_count += 1
            return True
        else:
            return False

    def play_shiritori(self):
        while True:
            if current_word := self.prompt_word():
                if self.play_word(current_word):
                    self.print_feedback(f'Yatta! You entered "{current_word}".')
                else:
                    self.print_feedback(f'You gave {self.word_count} words!')
                    return

sh = Shiritori(('apple', 'eggplant', 'tomato', 'okra', 'sitaw', 'bataw', 'patani'))
sh.play_shiritori()
```

[65♥ points]

[180 points]

- Does it violate the Single Responsibility Principle? why or why not?
 - Given this code, re-implement it using the Model-View-Controller architecture. In particular:
 - Ensure that the overall functionality remains the same; that is, a call to the controller’s run method (or whatever it’s called) will give the same user experience as the `play_shiritori` method above.
 - While you may write pseudocode and/or detailed sentences as your answer instead of actual Python code, ensure that its level of detail sufficiently covers everything being asked for. Unambiguous syntax errors will be tolerated.
3. The function `byenary_search` is intended to accept a closed integer interval $[l, r]$ ($l \leq r$), an integer y , and a function f , and looks for $x \in [l, r]$ such that $f(x)$ is equal to y . If any such x is found, the function returns x , otherwise, it returns `None`. A sample implementation and an assertion example is provided in Code Block B; note that there’s a behavioral requirement on the function f . (See [Appendix A](#) in case you are not familiar with `Callable`.) However, there is something wrong with the given implementation.

Code Block B Code for item 3

```
def cube(x: int) -> int:
    return x**3

def byenary_search(l: int, r: int, y: int, f: Callable[[int], int]) -> int | None:
    """
    The function `f` is required to be a pure function, and also strictly increasing.
    """
    while l < r:
        m = (l + r) // 2
        if f(m) < y:
            l = m
        elif f(m) > y:
            r = m
        else:
            return m
    return None

assert byenary_search(0, 3, 8, cube) == 2
```

- [40♥ points] (a) Provide at least one set of values for l , r , and y such that `byenary_search` fails to return the correct answer.
- [40♥ points] (b) Provide at least one set of values for l , r , and y such that `byenary_search` returns the correct answer.
- [70 points] (c) Fix the implementation of `byenary_search` and explain why your fix resolves the bug/s in the code.
- [81 points] 4. Consider the function and class hierarchy shown in Code Block C. (See Appendix A in case you are not familiar with `Callable`.)

Code Block C Code for item 4

```
def manip_twice(
    seq: Iterable[Pikachu], manip: Callable[[Pikachu], Pikachu],
) -> Iterable[Pikachu]:
    return (manip(manip(x)) for x in seq)

class Pokemon: ...
class Pikachu(Pokemon): ...
class Raichu(Pikachu): ...
```

Suppose the possible types of `seq` are:

- A. `list[Pokemon]` B. `list[Pikachu]` C. `list[Raichu]`

Suppose the possible types of `poke` are:

- | | |
|--|---|
| 1. <code>Callable[[Pokemon], Pokemon]</code> | 6. <code>Callable[[Pikachu], Raichu]</code> |
| 2. <code>Callable[[Pokemon], Pikachu]</code> | 7. <code>Callable[[Raichu], Pokemon]</code> |
| 3. <code>Callable[[Pokemon], Raichu]</code> | 8. <code>Callable[[Raichu], Pikachu]</code> |
| 4. <code>Callable[[Pikachu], Pokemon]</code> | 9. <code>Callable[[Raichu], Raichu]</code> |
| 5. <code>Callable[[Pikachu], Pikachu]</code> | |

For which pairs of types of `seq` and `poke` will the call `manip_twice(seq, poke)` type-check correctly? Format your answer as a table like so (where each “?” is either “**yes**” (if it will type-check) or “**no**” (if it will not type-check)):

	1	2	3	4	5	6	7	8	9
A	?	?	?	?	?	?	?	?	?
B	?	?	?	?	?	?	?	?	?
C	?	?	?	?	?	?	?	?	?

Note that for this item, incorrect answers will give point deductions! (A missing answer is not considered incorrect.) More specifically:

- leaving a cell blank gives you 0 points;
- putting in the correct answer (yes or no) gives you 3 points;
- putting in an incorrect answer gives you −4 points.

[50♥ points] 5. What is the output of Code Block D?

Code Block D Code for [item 5](#)

```
class Student:
    def __init__(self, name: str):
        self.name = name
        super().__init__()

    def label(self) -> str:
        return 'Student'

    def identifier(self) -> str:
        return f"{self.label():} {self.name}"

class UPStudent(Student):
    def __init__(self, name: str, student_no: str):
        self.student_no = student_no
        super().__init__(name)

    def identifier(self) -> str:
        identifier = super().identifier()
        return f"[{self.student_no}] {identifier}"

class DLSUStudent(Student):
    def __init__(self, name: str, salary: int):
        self.salary = salary
        super().__init__(name)

    def identifier(self) -> str:
        identifier = super().identifier()
        return f"{identifier} (salary {self.salary})"

class DCSStudent(UPStudent):
    def __init__(self, name: str, student_no: str):
        super().__init__(name, student_no)

    def label(self) -> str:
        return 'DCS Student'

    def identifier(self) -> str:
        identifier = super().identifier()
        return f"[DCS] {identifier}"

def some_students() -> list[Student]:
    return [
        Student('Piattos'),
        DLSUStudent('Nova', 125),
        UPStudent('Oishi', '1969-00012'),
        DCSStudent('Chippy', '1094-00012'),
    ]

for student in some_students():
    print(student.identifier())
```

6. Consider the code in Code Block E.

[60♥ points]

(a) Which SOLID principle(s) does it violate, and why?

[65 points]

(b) Rewrite the above code so that it follows SOLID principles.

[45 points]

7. Explain in detail how it is possible for an instance of StudentSorter as defined in Code Block F to arrive at a state that violates its stated invariant.

Code Block E Code for item 6

```
class CombinationLockVault:
    def __init__(self, initial_passcode: str):
        ...

class TimeLockVault:
    def __init__(self, hours: int):
        ...

class Bank:
    def __init__(self):
        self.vault = CombinationLockVault("00000")
        ...
```

Code Block F Code for item 7

```
@dataclass(frozen=True)
class Student:
    name: str
    section: str

class StudentSorter:
    """
    Each list in `_mapping` is guaranteed to contain only students
    with `section` equal to the key associated with the list.
    """
    def __init__(self, students: Sequence[Student]):
        self._mapping: dict[str, list[Student]] = {s.section: [] for s in students}

        for s in students:
            self._mapping[s.section].append(s)

    @property
    def mapping(self):
        return {k: self._mapping[k] for k in self._mapping.keys()}
```

- [40 points] 8. Consider the following attempts at type-annotating the same function. (See [Appendix C](#) in case you are not familiar with `tuple[T, ...]`.)

Implementation 1:

```
def lengths_of_palindromes(seq: tuple[str, ...]) -> tuple[int, ...]:
    return tuple(len(w) for w in seq if is_palindrome(w))
```

Implementation 2:

```
def lengths_of_palindromes(seq: Iterable[str]) -> Iterable[int]:
    return tuple(len(w) for w in seq if is_palindrome(w))
```

Implementation 3:

```
def lengths_of_palindromes(seq: tuple[str, ...]) -> Iterable[int]:
    return tuple(len(w) for w in seq if is_palindrome(w))
```

Implementation 4:

```
def lengths_of_palindromes(seq: Iterable[str]) -> tuple[int, ...]:
    return tuple(len(w) for w in seq if is_palindrome(w))
```

Which implementation is best, and why?

- [70 points] 9. `isinstance(<object>, <type>)` is a built-in Python function that returns `True` if the type of `<object>` is a *subtype* of `<type>`, or `False` otherwise. As an example, `isinstance(False, int)` returns `True` since `bool <: int`.

`isinstance` may be used to *narrow* the inferred type of an object initially inferred as type `T` into a subtype `S` by using it in conditionals, provided that `S <: T` and `isinstance(<object>, S)` is `True`.

Illustrate how `isinstance` can be used to create code that violates the *Open-Closed Principle* via a concrete scenario involving a class hierarchy, then sufficiently show how the violation

may be addressed by using an approach that does *not* employ `isinstance`. Your answer may be in the form of pseudocode and/or a detailed explanation for both cases.

[70 points] **10.** Consider the following classes, which are meant to represent Pokémon moves:

```
class Move:
    ...

class PsychicTypeMove(Move):
    ...

class ElectricTypeMove(Move):
    ...
```

We introduce a class called `Pikachu` which is meant to represent `Pikachu`:

```
class Pikachu:
    def choose_move(self, moves: list[Move]) -> Move:
    ...
```

Now consider the following classes, which are meant to represent other Pokémon by inheriting the `Pikachu` class:

```
class Raichu(Pikachu):
    def choose_move(self, moves: list[Move]) -> ElectricTypeMove:
    ...
```

```
class Mew(Pikachu):
    def choose_move(self, moves: list[PsychicTypeMove]) -> PsychicTypeMove:
    ...
```

```
class Zorua(Pikachu):
    def choose_move(self, moves: list[Move]) -> Move | None:
    ...
```

```
class Pichu(Pikachu):
    def choose_move(self, moves: list[Move] | None) -> ElectricTypeMove:
    ...
```

```
class Ditto(Pikachu):
    def choose_move(self, moves: list[Move | None]) -> Move:
    ...
```

Which of these five classes violate the Liskov Substitution Principle, and why?

[84 points] **11.** The *behavioral rules* of the *Liskov Substitution Principle* are as follows:

- (1) Preconditions must not be strengthened
- (2) Postconditions must not be weakened
- (3) Invariants must not be violated
- (4) The history rule must not be violated

For the classes of Code Block G listed below, identify **all** behavioral rules it violates. If the class does *not* violate any behavioral rules, explicitly state so. For each rule violation, explain why the code violates the rule. You may use the numbers 1 to 4 as shown above to more concisely refer to the behavioral rules.

1. StudentSubmission
2. AdminSubmission

3. PythonJudge
4. PypyJudge

12. You are in charge of improving the design of a donation campaign management app in which *donors* should only be allowed to *donate* to a campaign, while *campaign starters* should only be permitted to *withdraw funds* from it.

[65♥ points] (a) Identify one SOLID principle that is violated by the partial model design in Code Block H, and explain how the design violates it.

Code Block G Code for item **11**

```
class CodeLengthError(ValueError):
    pass

class Submission:
    """
    Raises a `ValueError` upon creation if `code` is
    empty or has length greater than 1000.

    `code` is guaranteed to stay the same upon creation.
    """
    def __init__(self, code: str, language: str):
        ...

class StudentSubmission(Submission):
    """
    Raises a `ValueError` upon creation if `code` is
    empty or has length greater than 500.

    `code` is guaranteed to stay the same upon creation.
    """
    ...

class AdminSubmission(Submission):
    """
    Raises a `CodeLengthError` upon creation if `code` is
    empty or has length greater than 1000.
    """
    ...

@dataclass
class Verdict:
    status: str

class Judge:
    def judge(self, submission: Submission) -> Verdict:
        """
        Returns a `Verdict` with `status` equal to one of: "AC", "WA", "TLE"
        """
        ...

class PythonJudge(Judge):
    def judge(self, submission: Submission) -> Verdict:
        """
        Returns a `Verdict` with `status` equal to one of: "AC", "WA", "TLE", "IR"
        `submission` must have `language == "Python"`.
        """
        ...

class PypyJudge(PythonJudge):
    def judge(self, submission: Submission) -> Verdict:
        """
        Returns a `Verdict` with `status` equal to one of: "AC", "WA"
        `submission` must have `language == "Python"`.
        """
        ...
```

Code Block H Code for item 12 (assume *CreditCardInfo* is defined as a Protocol)

```
class GoFundMeCampaign(Protocol):
    def donate(self, credit_card_info: CreditCardInfo, amount: int):
        ...

    def withdraw(self, amount: int):
        ...

class Donor(Protocol):
    def donate_to(self, campaign: GoFundMeCampaign, amount: int):
        ...

class CampaignStarter(Protocol):
    def __init__(self, campaign: GoFundMeCampaign):
        ...

    def withdraw_funds(self, amount: int):
        ...
```

[65 points]

- (b) Provide a fixed version that addresses the violation stated with an explanation of how the violation is addressed.

13. A *Pokémon generation* refers to a set of Pokémon games released around the same time. Each generation introduces new Pokémon and changes to existing Pokémon. As of this time of writing, there are nine Pokémon generations.

Gallade is a Pokémon that was introduced in *Generation IV* (a.k.a. *Gen 4*) with the *ability* called *Steadfast*. In Gen 5, an ability called *Justified* was made available to Gallade. From Gens 6-8, Gallade retained both abilities made available to it since Gen 5. In Gen 9, yet another ability called *Sharpness* was made available to Gallade.

For the subitems below, ensure that your solutions satisfy the *Open-Closed Principle*; adding support for future gens in which Gallade appears should *add* new code and *not* modify existing code.

[55 points]

- (a) Using *inheritance* and *abstract classes*, create a class hierarchy that has a superclass called *Gallade* and one subclass of *Gallade* per generation that *Gallade* appears in (named uniquely according to its associated generation).

Gallade and its subclasses must have a method `get_ability_set(self) -> set[str]` which returns a set of strings corresponding to all abilities available to *Gallade* during the generation associated with the class. Classes that have *exactly* the same behavior must use inheritance to minimize code duplication.

[55 points]

- (b) Create a class hierarchy (unrelated to the previous subitem) with a Protocol-based interface as its superclass with each implementing subclass uniquely representing a generation that *Gallade* appears in (i.e., Gens 4-9; classes must be named uniquely according to this) with information on the abilities available to it during its associated generation. Classes with the *exactly* the same behavior must use inheritance to minimize code duplication; outside these classes, inheritance must **not** be used.

Additionally, create a class called *Gallade* (unrelated to the previous subitem) which is *composed* of an object in the said hierarchy (via dependency injection with its constructor) that is used to dictate the return value of the `get_ability_set(self) -> set[str]` method of *Gallade*.



Bonus

[13♥ points]

14. What is the airspeed velocity of an unladen swallow?

合格しますように！！

A Annotations for Callables

Callables are objects that can be called, possibly on a set of arguments. Functions and methods are callables. Callables can be annotated using `collections.abc.Callable`.

A Callable annotation is of the form `Callable[[arg1_type, arg2_type, ...], return_type]`. For example, the annotation `Callable[[int], str]` signifies a function that takes a single parameter of type `int` and returns a `str`.

The following code shows a sample annotation for a function that accepts a function as one of its parameters.

```
from collections.abc import Callable

def n_hellos_with_dashes(n: int) -> str:
    return '-'.join('hello' for _ in range(n))

def print_triangle(n: int, f: Callable[[int], str]) -> None:
    for i in range(1, n + 1):
        print(f(i))
    for i in reversed(range(1, n)):
        print(f(i))

print_triangle(3, n_hellos_with_dashes)
```

```
hello
hello-hello
hello-hello-hello
hello-hello
hello
```

B Assignment expressions

The walrus operator `:=` is a syntactic sugar used to produce assignment expressions—that is, expressions which also assign variables. The code “`VAR := EXPR`” is roughly equivalent to the expression “`EXPR`”, but with the added effect of assigning its result to the variable “`VAR`”.

For example, it can be used to simplify the code

```
res = f(x, y)
if res:
    print("res is okay!", res)
else:
    res = g(w)
    if res:
        print("res is semi-okay", res)
    else:
        print("res is not okay", res)
```

to

```
if res := f(x, y):
    print("res is okay!", res)
elif res := g(w):
    print("res is semi-okay", res)
else:
    print("res is not okay", res)
```

It has been designed to only be usable in some limited situations because of its potential for abuse/bugs.

C Variable-length tuple annotations

To denote a tuple which could be of *any* length, and in which all elements are of the same type `T`, use `tuple[T, ...]`.